

XB-Muse: Practical Multiuser Dynamic Searchable Symmetric Encryption for Adaptive Revocation

Qin Jiang^{1b}, Xu Yang^{1b}, Sifan Chen, Saiyu Qi^{1b}, Fuyuan Song, and Zhangjie Fu^{1b}, *Member, IEEE*

Abstract—Dynamic searchable symmetric encryption (DSSE) schemes support keyword search queries on encrypted dynamic datasets with add-or-delete operations stored on an untrusted remote server. Multiuser DSSE (MUDSSE) further considers multiple users to access the encrypted dataset. Most works of MUDSSE focus on how to promise forward and backward privacy of queries. However, the problem where the data owner can not revoke the deleted encrypted data on the encrypted dataset adaptively and efficiently is not sufficiently considered. To solve this problem, we propose a new MUDSSE scheme named X-MUSE, extending cryptographic primitives symmetric revocable encryption (SRE) to design a new searchable encryption with optimal search time for the deletion operation. Furthermore, to minimize communication size and counter new integrity threats raised by malicious clients, we extend X-MUSE to design a new MUDSSE named B-MUSE by integrating blockchain-based technology. Our evaluation confirms that our schemes have practical search performance and lower storage costs on both the server and the client side compared to the state-of-the-art.

Index Terms—Adaptive revocation, blockchain, dynamic multiuser searchable symmetric encryption, forward and backward privacy.

I. INTRODUCTION

DATA sharing and outsourcing have become prevalent for individuals and businesses. For example, cloud service platforms, such as Google Health, and Microsoft Health Vault, are available to all participants to conveniently and efficiently share health information with different medical institutions or individuals. To protect data security and privacy by outsourcing to the untrusted third-party cloud, one representative work such as searchable encryption (SE) is to enable keyword search over encrypted data while protecting the confidentiality of the data and search query.

Received 25 January 2025; revised 20 March 2025; accepted 7 April 2025. Date of publication 16 April 2025; date of current version 9 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62302229; in part by the Startup Foundation for Introducing Talent of NUIST under Grant 2023r043; in part by the National Natural Science Foundation of China under Grant 62302230, Grant U22B2062, and Grant 62172232; and in part by the National Key Laboratory of Science and Technology on Space Microwave under Grant HTKJ2023KL504006. (*Corresponding author: Zhangjie Fu.*)

Qin Jiang, Fuyuan Song, and Zhangjie Fu are with the Engineering Research Center of Digital Forensics, Ministry of Education, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: qinj@nuist.edu.cn; fysong@nuist.edu.cn; wwwfzj@126.com).

Xu Yang, Sifan Chen, and Saiyu Qi are with the School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an 710049, China (e-mail: yangxu@stu.xjtu.edu.cn; 2193469471@qq.com; saiyu-qi@xjtu.edu.cn).

Digital Object Identifier 10.1109/IIOT.2025.3561287

Recently, a stronger cryptographic primitive multiuser SE (MUSE) enables the data owner to share ciphertexts with multiple recipients. Specifically, the building blocks of MUSE are symmetric SE (SSE) [1], [2], [3] and public-key SE (PKSE). PKSE [4], [5], [6] is naturally used in multiuser scenarios but incurs inherent performance limitations. SSE usually has high performance, but may not be easy to enable access control functionalities in multiuser scenarios. Specifically, removing the user's search permission may require the data owner to download the entire encrypted index, re-encrypt it with a new key that the user does not know and then send it back to the server. This process requires a large amount of bandwidth and processing power from the server. Recently, Wang and Chow [7] proposed hybrid SE (HSE), which elegantly combines SSE and PKSE to take advantage of the advantages of both models: 1) efficiency by SSE and 2) multiwriter capability by PKSE. Le et al. [8] proposed MUSES focused on detecting security weaknesses in both models, including keyword-guessing vulnerabilities and pattern leakages. Despite its merits, HSE and MUSE both inherit efficiency weaknesses due to the building block PKSE. On the other hand, dynamic SE (DSE) schemes have leakage considerations related to updates, i.e., do not reveal any connection of newly modified entries to previous ones, which is called forward privacy (FP), or do not reveal information about deleted entries during queries after the deletion, which is called backward privacy. DSE schemes that protect against both these leakages are called forward-and-backward private schemes [9]. Q- μ SE [10] enables users to search ciphertext while protecting forward-and-backward privacy with high performance in terms of computing costs by using the lightweight block SSE. However, it still suffers from shortcomings in communication and storage costs.

Can we design a new multiuser SSE scheme that not only supports multireader similar to Q- μ SE, but also addresses the inherent communication and storage limitations of existing paradigms, while achieving adaptive revocation of the document access control?

We answer the above question affirmatively by proposing two new multiuser SSE schemes named X-MUSE and B-MUSE, which promise forward and backward privacy. Our main idea is to use the advancements in hardware-assisted trusted execution environments (TEEs), such as Intel SGX, a set of instruction extensions that provide a secure memory region, and a cryptographic primitive symmetric revocable encryption (SRE) [11], which is a SSE scheme based on symmetric encryption, and uSE [10], a generic transformation

guideline for SSE scheme adoption in the multiuser scenario. However, it is not easy to combine these primitives to design a new SSE scheme. We begin by giving SRE's overview and then outline the challenges.

Brief Overview of SRE: SRE [11] is a single-client dynamic encrypted search scheme that promises forward and backward privacy. Its goal is to allow the data owner to carry out the deletions locally while consuming a low storage request and promise backward-private properties in a noninteractive way. To revoke document access control in one step rather than in an incremental manner, SRE first uses a bloom filter to store the tags associated with the deleted document. For revocation, on the entries of the Bloom filter with the value "1" corresponding to the tags need to be revoked. SRE creates a punctured secret key. With this key, SRE computes the pseudo-random function value for an unrevoked tag. The ciphertext associated with the tag can be decrypted using this PRF value.

Extending SRE to the multiuser scenario is not an easy work, as one of the building blocks of SRE is a symmetric encryption scheme consisting of a multiple puncturable PRF. First, when the user is directly given access to the symmetric key, they run the risk of being corrupted or working with the adversary in control of the server and the user may share the key with the adversary, which leads to serious security issues. Second, because the server can use the previously revoked secret key to decrypt the nondeleted document indices inserted in the future, SRE requires the user to refresh the encryption after every search. However, frequent key refreshing incurs inherent communication performance limitations between the data owner and the user. Third, the user is not always online, the fresh data structure can not be sent to the user immediately.

Challenge 1: How to protect the critical private key from corrupted users to ensure secure data search and dynamic update?

Approach: Our first idea is to delegate the critical private key generated by the multipuncturable PRF to the enclave created by the Intel SGX [12] instead of sharing the symmetric key with the users directly. Intel SGX is a set of instruction extensions that aim to create a trusted memory named enclave on the untrusted server. In SRE and incremental puncturable encryption in both Janus [9] and Janus++ [13], the data owner produces a fresh key component and outsources it to the user whenever a deletion happens. The reason is that the revoked secret key is leaked to the server directly. Using an enclave can avoid the leakage of the secret key, which successfully avoids frequent interaction between the data owner and client to refresh the key.

Challenge 2: How to promise the untrusted server to send the latest version rather than the historical version data structure to the user, namely an anti-replay attack?

Approach: We first affirm that the critical data structure needed to load from the server to the user in our first scheme X-MUSE. We observe that the compressed deletion list D is the critical data structure, namely a Bloom filter. D is used to compute the revoked secret key on the client side, which is used to decrypt the nondeleted identifiers. The user needs to download D from the server. However, as D is stored on the untrusted cloud server, the server may send the historical version of D to the user, which is called the replay attacks.

To prevent the replay attack, we modify X-MUSE slightly to design a new scheme named X-MUSE*. Specially, we design a list $Delcnts$ which stores an encrypted counter to record the deletion times cnt' . When the user verify the freshness of D , it downloads $Delcnts$ from the server to get the fresh cnt' and compares it with the counter associated with D .

Challenge 3: How to reduce the communication cost between the data owner and the cloud server and prevent the server from returning the core data structure to the user maliciously?

Approach: When the user requests a search to the server, the server needs to send the compressed data structure which is the bloom filter to the client. The frequency of iterations between the client and the server leads to high communication costs. Meanwhile, the cloud server may tamper with the bit in the bloom filter, which leads the client to compute the mistake search token. Although recent verifiable SE schemes can resist a malicious server attack [14], [15], [16], they do not focus on multiuser scenarios. In this article, we propose a scheme named B-MUSE which provides an on-chain revocation status service by employing blockchain technology. In particular, the data owner uploads the revocation list to blockchain nodes, which updates the global revocation status for all data users. When a user wants to send a query to the server, she first queries the blockchain nodes for the revocation status to compute the search token and the revocation secret key. B-MUSE can not only provide a batch revocation service but also promise the correctness of core data structure by using blockchain.

The main contributions of this article can be summarized as follows.

- 1) Toward high storage costs on the server and the client and high communication costs challenges, our first construction X-MUSE is the first MUSSE that can adaptively delete the files against user-server collusion and achieves fine-grained access control for the user. In addition, we design a scheme named X-MUSE* to defend against the replay attack based on X-MUSE.
- 2) Regarding the high latency for deleting the key-value pairs in the encrypted datasets and malicious cloud server, Our second construction named B-MUSE, is the second MUSSE that can adaptively delete the files for batch and prevent the cloud server tampering with the revocation status by employing blockchain.
- 3) We compare our works with the state of the arts in Table I and prove the security of our X-MUSE and B-MUSE under the real and ideal paradigm. We implement our SSE schemes. Experiment results show that our schemes can fetch all matched results in a query with a negligible false positive and with short latency, and achieve better communication and storage efficiency compared with the state of the art scheme [10].

II. SYSTEM OVERVIEW

A. System Model

As shown in Fig. 1, our system model includes three types of entities: 1) data owner; 2) data users; and 3) cloud server.

TABLE I
COMPARISON WITH RELATED WORKS

Schemes	Communication			Computation		Client Storage	Multi-user Setting
	Rounds	Search	Update	Search	Update		
<i>Aura</i> [11]	1	$\mathcal{O}(n_w)$	$\mathcal{O}(1)$	$\mathcal{O}(n_w)$	$\mathcal{O}(1)$	$\mathcal{O}(Wd)$	x
<i>O-μSE</i> [10]	$\mathcal{O}(\log(W))$	$\mathcal{O}(a_w)$	$\mathcal{O}(\log(N_w))$	$\mathcal{O}(N_w)$	$\mathcal{O}(\log(N_w))$	$\mathcal{O}(\log(N_w))$	✓
<i>Q-μSE</i> [10]	1	$\mathcal{O}(N_w)$	$\mathcal{O}(1)$	$\mathcal{O}(N_w)$	$\mathcal{O}(1)$	$\mathcal{O}(N_w)$	✓
<i>χ-MUSE</i>	1	$\mathcal{O}(a_w)$	$\mathcal{O}(1)$	$\mathcal{O}(a_w)$	$\mathcal{O}(1)$	$\mathcal{O}(A_w)$	✓
<i>B-MUSE</i>	1	$\mathcal{O}(a_w)$	$\mathcal{O}(1)$	$\mathcal{O}(a_w)$	$\mathcal{O}(1)$	$\mathcal{O}(A_w)$	✓

([†]In this table, for a keyword w , a_w is the total number of inserted entries matching w , d_w denotes the number of deleted entries matching w and $d = \max_w d_w$, n_w is the size of search result matching w and $n_w = a_w - d_w$ and $N_w = a_w + d_w$. W denotes the total number of keywords of all documents, and $A_w = \max_w a_w$.)

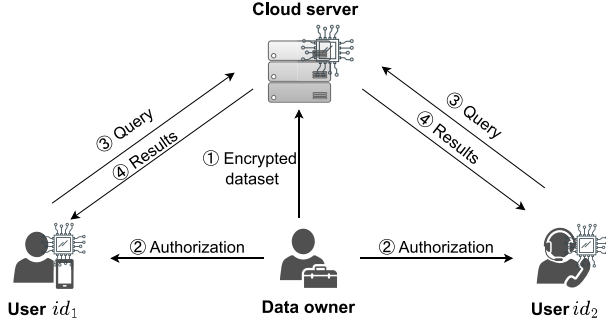


Fig. 1. System model.

- 1) **Data Owner:** The data owner generates secret keys to encrypt dataset and establishes indexes to over the dataset. She also generates secret keys named master keys for authorized data users so that they can generate search tokens for search queries. She can also revoke the access control for the dataset by adding the tags associated with the deleted encrypted identities in a revocation list.
- 2) **Data User:** For a query, the data user computes the search token and uploads it to the cloud server and gets the query results.
- 3) **Cloud Server:** The cloud server stores the encrypted index and dataset submitted by the data owner. When the cloud server receives a search token from a data user, it searches on the encrypted index and returns the query results to the data user.

B. Threat Model

We assume that the data owner is trust-worthy and the cloud server is malicious. Specifically, the data owner faithfully builds the encrypted indexes, master keys and revocation list. For the data users, they may be compromised or corrupted. Specifically, they may be dishonest about the search token generation and collaborate with the server in order to extract information about part of plaintext about the encrypted datasets or the plaintext of the search token. For the cloud server, he could maliciously manipulate the data structure sent by the data owner due to commercial interests or security vulnerabilities. In addition, he might attempt to learn the content of the search tokens based on the updated operations.

C. Design Goals

We aim to design an MUDSSE scheme that satisfies the following goals.

- 1) **Timeliness and Freshness:** The data owner can adaptively delete the encrypted data on the cloud server by a revocation list instead of deleting it on the cloud server directly, which promises timeliness. After the data owner uploads the revocation list, the client can get a fresh revocation status to generate the search token, which promises freshness.
- 2) **Privacy:** When updating or querying the encrypted datasets, the client does not reveal the details of the server it intends to access, which promises forward and backward privacy in multiclient.
- 3) **Low Communication Costs:** When the user requests a query, the search result between the client and the server is optimal.
- 4) **Low Storage Costs:** When the data owner wants to remove some data on the server, the server can remove the corresponding encrypted data in the batch.

D. Function Definition

A dynamic multiuser searchable symmetric encryption scheme consists of five algorithms, such as Setup, Enroll, Share, Update, Search, and UnShare. These algorithms are executed between a data owner, a server and several users.

- 1) **Setup**($1^\lambda; N; |W|; |U|; D$): Takes security parameter, size of distinct keyword set W , total size of dataset N , a set of initial files D and total number of users U consisting of tuples $id, Kw(id)_{id \in D}$ as input, it outputs the master secret key K of the owner and the encrypted version C of D to the server.
- 2) **Enroll**($1^\lambda, u$): In this algorithm, it inputs the security parameter λ and outputs the user key K_u and sends K_u to the user u .
- 3) **Share**($K, u, id, kw(id), mod$): Inputs data owner master key K , file identifier id , user u , set of keywords $kw(id)$ in file id and mode mod that can be add or share operation, it outputs the encrypted version C' of file id and a new or existing key K_{id} for the file id and the auxiliary data aux_u for user u , which is privately shared with user u .
- 4) **Update**($K, id, WList, op, aux_{uu \in U}, K_{uu \in U}$) is an algorithm for inserting or removing a list of keywords to or from an existing file. The data owner takes the secret key K , the keyword list $WList$ contained in id , a file identifier id , user's auxiliary information $aux_{uu \in U}$ and outputs the updated ciphertext C' and the updated version $aux'_{uu \in U}$.
- 5) **Search**($w, K_u, aux_u, C, Uset$): For the search algorithm, the user takes the search keyword w , the secret key K_u and the auxiliary data aux_u and then outputs the search

token. The server takes the encrypted dataset C and the authorization list $uSet$ and the search token as the inputs and then outputs the search results.

- 6) $Unshare(K, u, id, Kw(id), aux_u)$: Takes master key K , user u , set of keywords $Kw(id)$, file identifier id and the user's auxiliary data, it revokes the user access to id as input, it updates the ciphertext dataset C , the updated set of authorization $uSet'$ and the updated authorization data aux'_u .

E. Security Definitions

Definition 1 (Sim-Adaptive Security): A multiuser searchable symmetric encryption scheme $\pi = \{\text{Setup}, \text{Update}, \text{Share}, \text{Search}\}$ is adaptive secure with respect to Leakage function L , if for any PPT adversary A that issues a polynomial number of queries q, t there exists a PPT simulator S such

$$|\Pr[\text{Real}_A^\pi(1^\lambda) = 1] - \Pr[\text{Ideal}_{A,S,L}^\pi(1^\lambda) = 1]| \leq \text{negl}(1^\lambda)$$

where $\text{negl}(1^\lambda)$ is a negligible function in 1^λ . The probabilistic experiments $\text{Real}_A^\pi(1^\lambda)$, $\text{Ideal}_{A,S,L}^\pi(1^\lambda)$ are as follows.

$\text{Real}_A^\pi(1^\lambda)$: The challenger runs $\text{Setup}(1^\lambda)$ to generate a secret key SK and sends (initially empty) EDB to A . A outputs a set of documents $D = \{d_1, d_2, \dots, d_n\}$ and sends D to the challenger. Then, A makes a polynomial number of adaptive queries q with input w or update queries with input (op, w, d) to the challenger. After that, the challenger calculates $\tau \leftarrow \text{Search}(w)$ and sends τ to A or calculates $R \leftarrow \text{Update}(op, w, d)$ and then sends R to A , where R is a set of identifiers. Finally, A returns a bit b which is the output of the experiment.

$\text{Ideal}_{A,S,L}^\pi(1^\lambda)$: The adversary A outputs a set of documents $D = \{d_1, d_2, \dots, d_n\}$. Using the setup leakage function L_H^{Sp} , the simulator S generates in inertial empty EDB and sends it to A . Then, A creates a polynomial number of adaptive queries q with input keyword w or update queries with input (op, w, d) to the simulator S . Then, using L_H^{Srch} , S calculates τ and sends τ to A . Using L_H^{updt} , S calculates R and sends R to A , where the R is a set of identifiers. Finally, A returns a bit b which is the output of the experiment.

III. PRELIMINARIES

A. Symmetric Revocable Encryption

SRE [11] is a cryptographic primitive that can help to achieve noninteractive deletion.

- 1) $SRE.KGen(1^\lambda)$: With the security parameter λ , it outputs the master secret key $msk = (sk, D)$ where sk is the secret key used for generating the revoked secret key and D is a compact data structure that is a bloom filter which will contains the tags associated with deleted files.
- 2) $SRE.Enc(m, msk, t)$: Inputs message m , the secret key msk and the tag t associated with m and outputs the ciphertext ct .
- 3) $SRE.Comp(D, t)$: Inserts the revoked tag t into the bloom filter D using hash functions and outputs the new bloom filter D .

- 4) $SRE.ckRev(sk, D)$: Takes sk and D as input and outputs the revoked secret key sk_R .
- 5) $SRE.Dec(sk_R, ct)$: If the related tag t associated with ct is not in D , it decrypts ct via sk_R .

B. Blockchain

Blockchain has recently raised major attention in both industries and academia. Blockchain is a distributed database. Smart contracts were not widely used in the initial era of the blockchain, mainly represented by the Bitcoin system. One of the advantages of the smart contract is that it is characterized by its high timeliness and decentralized contract formulation, which eliminates the need for third-party authority participation.

C. Intel SGX

Intel SGX [12] is a set of instruction extensions. Using these instruction extensions can create a trusted memory region called an enclave on the untrusted memory of the cloud server. The code and data run in the enclave can not be visited directly by the procedure out of the enclave including the operating system. As the memory created by the intel SGX usually is limited even Intel TDX [17], the procedure in the enclave needs to visit the data stored on the untrusted memory. On the other hand, the procedure on the untrusted memory may need to interact with the algorithm within the enclave. SDK provided by Intel SGX provides two secure access channels named *ecall* and *ocall*. Developers can use *ecall* to let the programs in the untrusted space access the data stored in the enclave and use *ocall* to let the programs in the trusted space access the data stored in the untrusted memory. The interaction between the enclave and the untrusted memory can cause efficiency and security problems. As the enclave consists of a contiguous physical memory pages frequently interacts with the untrusted memory and the underlying algorithm of the enclave checks the integrity of the data and has the encryption and decryption operations for every interaction. Moreover, the interaction between the enclave and the untrusted memory suffers from side channel attacks such as page fault attack [18] when the data size is huge. However, when the data size is small, it can reduce the interaction, and the probability of a successful side-channel attack is greatly reduced.

D. Forward and Backward Privacy in MUSSE

Common Leakage Functions: Akin to [9], $sp(w, u)$ represents the search pattern, which denotes the search sequences belonging to the keyword w of the user u . In this section, the leakage functions including the setup leakage L_H^{Sp} and the search leakage L_H^{Srch} , the share leakage L_H^{Shr} and the update leakage L_H^{updt} are defined as follows.

Forward Privacy (FP): FP ensures an update is not linked with any previous queries. To capture the FP in MUSSE, it is necessary to define some helper functions *ShrLeakage* and

UpdtLeakage for timestamp t

$$\begin{aligned} \text{ShrLeakage}(u, id, t) &= \begin{cases} Kw^t(id) & u \in C \\ |Kw^t(id)|, & \text{otherwise} \end{cases} \\ \text{UpdtLeakage}(u, WList) &= \begin{cases} WList & u \in C \\ |WList|, & \text{otherwise} \end{cases} \end{aligned}$$

where $Kw^t(id)$ is the keywords contained in document id , $WList$ denotes the keyword list, C denotes the set of the corrupted users. if the user u is corrupted, it leaks all the keywords contained in id during the share operation, while it leaks all the keywords contained in the $WList$ during the update operation.

Definition 2: An L -adaptively secure MUSES is forward-private in the presence of corrupted users, if the Share, Update leakage functions (L^{Share}, L^{Update}) can be written as

$$\begin{aligned} L_H^{Shr}(u, id, Kw^t(id), \delta) &= L'(u, id, \text{ShrLeakage}(u, id, t)) \\ L_H^{Updt}(u, KwList, op, \delta) &= L'(id, \text{UpdtLeakage}(u, KwList) \\ &\quad op, \text{AccessList}(id)) \end{aligned}$$

where L' is a stateless function.

Backward Privacy (BP): BP ensures that a search for w can not get the identifiers which have been previously deleted associated with w . Three types of backward privacy for single-keyword search are defined in [9]: Type-I backward privacy, Type-II backward privacy and Type-III backward privacy. Type-I backward privacy has strongest security while Type-III backward privacy has the weakest security. Type-I backward privacy allows leaking the identifiers of the keyword w queried and the timestamps of inserting these identifiers into the database. Type-II backward privacy additionally leaks operation types of all updates on the queried keyword w and the timestamps of the updates, besides the information leaked in Type-I. Type-III further leaks which deletion operation canceled precisely which addition compared to Type-II. To capture the backward privacy in MUSSE, we define two helper functions *TimeDB* and *Update* for timestamp t . *TimeDB* returns all (timestamp, identifier) pairs of keyword w that have been added by either an update or a share operation to DB and have not been deleted or unshared afterward for user u .

$$\begin{aligned} \text{TimeDB}(w, u) &= (t, id) | (id, Kw(id), u, t) \in Q : w \in Kw(id) \text{ and} \\ &\quad (id, KwList, share, UserList, t) \in Q: \\ &\quad w \in KwList \wedge u \in UserList \\ &\quad \text{and } \forall t' (id, KwList, unshare, UserList, t') \notin Q: \\ &\quad w \in KwList \wedge u \in UserList \wedge t' < t \\ &\quad \text{or } (id, Kw(id), add, u, t) \in Q: w \in KwList \\ &\quad \text{and } \forall t' (id, Kw(id), del, u, t') \notin Q: \\ &\quad w \in KwList \wedge t' < t). \end{aligned}$$

Update return all the timestamps of keyword w for user u of sharing and update operation

$$\begin{aligned} \text{Update}(w, u) &= (t^{share} || t^{unshare} || t^{add} || t^{del}) | \\ &\quad (id, Kw(id), u, t^{share}) \in Q : w \in Kw(id) \end{aligned}$$

TABLE II
NOTATIONS

Notation	Meaning
N_{key}	number of different keywords in the dataset D
D	Compressed data structure
$ W $	number of keywords
H	the set of hash functions
K	the secret key
sk_R	the revoke secret key
$IDList$	the list of document identifies
$\$$	random select a value from a set
λ	security parameter
$ U $	number of users
C	the set of corrupted users
Q	the list of entries in a sequence of operations
AccessList	the list of authorized users

$$\begin{aligned} &\text{or}(id, Kw(id), u, t^{unshare} \in Q: w \in Kw(id)) \\ &\text{or}(id, KwList, add, UserList, t^{add}) \in Q \\ &\text{or}(id, KwList, del, UserList, t^{del}) \in Q: \\ &\quad w \in KwList \wedge u \in UserList \end{aligned}$$

$$\text{SrchLeakage}(w, u) = \begin{cases} w, & u \in C \\ \perp, & \text{otherwise.} \end{cases}$$

Definition 3: An L -adaptively secure MUSES is backward-private in the presence of corrupted users, if the search and update leakage functions L_H^{Srch} can be written as

$$\begin{aligned} L_H^{Srch}(w, K_u; EDB) &= L'(U, \text{TimeDB}(w, u) \\ &\quad \text{Update}(w, u), \text{SrchLeakage}(w, u)) \end{aligned}$$

where L' is a stateless function.

IV. CONSTRUCTIONS

In this section, we are ready to present our first construction *X-MUSE*. We first describe the main idea and overview of *X-MUSE* and then provide details about the algorithm of the scheme. Then, we design *X-MUSE** to defend against the relay attack based on *X-MUSE*. In particular, the pseudo-codes in blue are run in the enclave created by intel SGX. The frequency notations are shown in Table II.

A. Intuition and Overview

To achieve FP and backward privacy, the communication costs between the client and the server in $O\text{-}\mu SE$ [10] and $Q\text{-}\mu SE$ [10] are high. Meanwhile, to achieve backward privacy II, the server cannot know the operation of deletion or addition updates by the client, which leads to high storage on the server side as the server cannot physically remove deleted files. To solve these problems, we design a new MUDSSE by employing a TEE and SRE. In Fig. 2, the flow of the system involves 4 steps. At step 1, the data owner computes the index and encrypts the index and the dataset and then uploads the encrypted index and encrypted dataset to the cloud server. This step completes the **Setup** protocol of our scheme. At step 2, the data owner uses the SGX attestation feature to authenticate the enclave and establish a secure channel with the enclave. The data owner then provisions a secret key sk to the enclave of the corresponding user through this channel.

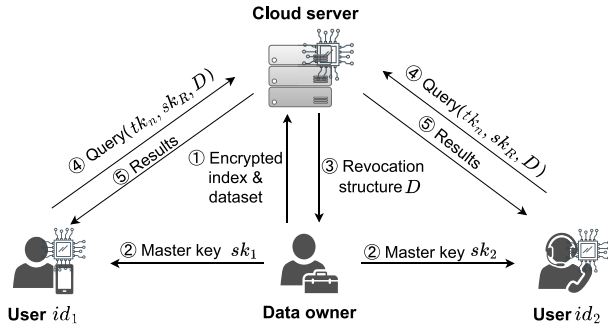


Fig. 2. Overview of X-MUSE.

Algorithm 1: Setup($1^\lambda, N, |W|, |U|$)

```

// Data owner:
1  $K_c, K_s \xleftarrow{\$} \{0, 1\}^\lambda$ 
2  $\text{DictW}, \text{UserKeys}, \text{Queues} \leftarrow \text{empty map}$ 
3  $\text{AccessList} \leftarrow \text{empty map of lists}$ 
4  $\delta \leftarrow (\text{AccessList}, \text{UserKeys})$ 
5  $\text{EDB}_{\text{cache}} \leftarrow \emptyset, \text{MSK}, \text{C}, \text{D} \leftarrow \perp$ 
6  $\text{EDB} \leftarrow (\text{DictW}, \text{Queues}, \text{EDB}_{\text{cache}}, \text{AccessList})$ 
7 Send EDB to the server

```

Algorithm 2: Enroll($1^\lambda, u$)

```

// Data owner:
1  $K_u, K'_u, K_t \xleftarrow{\$} \{0, 1\}^\lambda$ 
2  $\text{UserKeys}[u] \leftarrow K_u, K'_u, K_t$ 
3 Send UserKeys[u] to the user  $u$  via the secure channel

```

This step completes the **Enroll** protocol of our scheme. When the data owner wants to add/delete a (w, id) pair, then the data owner initializes or updates the data structure named **D** stored on the cloud server for deletion (see step 3). This step completes the **Share&Revoke** and **Update** protocols of our scheme. At step 4, the client wants to search documents matching a query keyword **D**, she first generates the revoked secret key sk_R using **D** and generates the search token. At step 4, the cloud server sends the search results to the user.

B. X-MUSE: Intel SGX-Based MUDSSE

Setup: Algorithm 1 shows the details of **Setup**, which includes data structure initialization. In the beginning, on input security parameter λ , the data owner generates a secret key K_c , which is used for encrypting the files by the symmetric encryption (we omit the initial file encryption from setup to simplify the presentation). Then, it initiates two empty maps **DictW** and **UserKeys**. **DictW** is used to store encrypted key-value pairs. **UserKeys** is used to store users' secret keys. **AccessList** is used to store the user's access list which contains the file identifiers and can be used by the server for access control during file retrieval. **EDB_{cache}** is used for store the search result for efficiency. **MSK** is used to store the user's key. **D** is used to store the revocation list and is a compact structure named bloom filter.

Enroll: Algorithm 2 shows that the user u enrolls for the query. The data owner generates the secret key for the user u .

Algorithm 3: Share&Revoke($K, u, id, kw^i(id), mod, \delta$)

```

// Data owner:
1 if  $mod = \text{share}$  then
2    $\text{AccessList}[id] \leftarrow \text{AccessList}[id] \cup \{u\}$ 
3  $\text{Keyvalues} \leftarrow \perp; \text{cntEnc} \leftarrow \perp; (K_u, K'_u, K_t) \leftarrow \text{UserKeys}[u]$ 
4 for each  $w$  in  $KW^i(id)$  do
5   if  $\text{FileCnts}[u][w] = \perp$  then
6      $\text{FileCnts}[u][w] = 0; (sk, D) \leftarrow \text{SRE.KGen}(1^\lambda)$ 
7      $\text{MSK}[u][w] \leftarrow sk; \text{D}[u][w] \leftarrow D$ 
8      $t \leftarrow F_{K_t}(w, id)$ 
9   if  $mod = \text{share}$  then
10     $\text{FileCnts}[u][w] ++; \text{cnt} \leftarrow \text{FileCnts}[u][w]$ 
11     $\text{cntEnc} \leftarrow \text{cntEnc} \cup \text{Enc}_{K'_u}(w, \text{cnt})$ 
12     $sk \leftarrow \text{MSK}[u][w]$ 
13    send  $sk$  to the enclave of user  $u$  privately
14     $D \leftarrow \text{D}[u][w]; \text{msk} \leftarrow (sk, D)$ 
15     $ct \leftarrow \text{SRE.Enc}(\text{msk}, id, t)$ 
16     $\text{val} = (ct, t); \text{addr} = G_{K_u}(w, \text{cnt})$ 
17     $\text{Keyvalues} \leftarrow \text{Keyvalues} \cup \{(addr, \text{val})\}$ 
18   else
19     if  $u.type$  is anti-replay attack then
20       if  $\text{FileDelCnts}[u][w] = \perp$  then
21          $\text{FileDelCnts}[u][w] = 1$ 
22       else
23          $\text{FileDelCnts}[u][w] ++$ 
24          $\text{cnt}' \leftarrow \text{FileDelCnts}[u][w]$ 
25          $\text{DelCntDiffs} \leftarrow \text{DelCntDiffs} \cup \{(u, \text{Enc}_{K'_u}(w, \text{cnt}'))\}$ 
26          $\text{addr} = G_{K_u}(w, \text{cnt}')$ 
27       else
28          $\text{addr} = G_{K_u}(w)$ 
29          $D \leftarrow \text{D}[u][w]; D \leftarrow \text{SRE.Comp}(D, t); \text{D}[u][w] \leftarrow D$ 
30          $\text{Revoketags}[u][w] \leftarrow (addr, D)$ 
31 if  $mod = \text{share}$  then
32   send  $\text{cntEnc}, \text{Keyvalues}, (u, id)$  to the server
33 else
34   send  $(u, \text{Revoketags}[u], \text{DelCntDiffs})$  to the server
// Server:
35 if  $mod = \text{share}$  then
36    $\text{AccessList}[id] \leftarrow \text{AccessList}[id] \cup \{u\}$ 
37   Insert all  $(addr, \text{val})$  in Keyvalues into Dictw
38   Insert  $(u, \text{cntEnc})$  into Queue[u]
39 else
40   Insert all  $(u, \text{Revoketags})$  into Revtag[u]
41   Insert all  $(u, \text{DelCntDiffs})$  into FileDelCnts[u]

```

Share and Revoke: Algorithm 3 shows details about how to share or revoke documents to the user u (lines 1 and 2). In the beginning, if the mod is sharing a document, then the data owner authorizes the access control of the document id to the user u . The data owner then gets the keys from **UserKeys**[u] (line 3). For each keyword w contained in the document id , it checks whether the keyword counter $\text{FileCnt}[u][w]$ is initialized or not. If not, the data owner calls the function **SRE.KGen** to generate the master key msk , which contains a secret key sk stored in $\text{MSK}[u][w]$ and a compact data structure D stored in $\text{D}[u][w]$ and initializes $\text{FileCnt}[u][w]$ to be 0, and $\text{FileCnt}[u][w]$ can help the scheme promise FP (lines 5–7). Compute a tag $t = F_{K_t}(w, t)$ for each encryption identifier id (line 8). For sharing a pair (w, id) with a user u , the data owner then increments the keyword counter and stores its current value in cnt and stores its encryption value in cntEnc (lines 10 and 11). After that, it computes the $(addr, \text{val})$ pair and then sends this pair stored in **Keyvalues** to the server (lines 12–17). Specifically, the ciphertext val of id is computed by **SRE.Enc** (lines 15 and 16). addr is computed by a pseudorandom

Algorithm 4: update($K, id, WList, op, \delta$)

```

// Data Owner:
1 userList  $\leftarrow$  AccessList[id]
2 Keyvalues  $\leftarrow \perp$ ; Cnt  $\leftarrow \perp$ 
3 for each  $u$  in userList do
4    $(K_u, K'_u, K_t) \leftarrow UsersKeys[u]$ 
5   for each  $w$  in WList do
6     lines 5-8 in Algorithm 3
7     if  $op=add$  then
8       lines 10-16 in Algorithm 3
9       Keyvalues  $\leftarrow Keyvalues \cup \{(addr, val)\}$ 
10    else
11      lines 19-29 in Algorithm 3
12      Revoketags  $\leftarrow Revoketags \cup \{(addr, D)\}$ 
13  if  $OP=add$  then
14    send  $u, cntEnc, Keyvalues$  to the server
15    send  $sk$  to the users' enclave privately
16  else
17    send  $u, Revoketags, DelCntDiffs$  to the server
// Server:
18 if  $OP=add$  then
19   Insert all  $(addr, val)$  in Keyvalues into Dictw
20   Insert  $(u, cntEnc)$  into Queue[ $u$ ]
21 else
22   Insert all  $(u, Revoketags)$  into Revtag[ $u$ ]
23   Insert all  $(u, DelCntDiffs)$  into FileDelCnts[ $u$ ]

```

function G (line 16). On the server side, the server adds u to **AccessList**[id] and stores $(addr, val)$ in **Dictw** and stores $cntEnc$ to the **Queue**[u]. On the other side, when the data owner revokes a user's access to a document id , the data owner inserts the tag associated with (w, id) into a compact data structure named D and then stores D with an address named $addr$ in an array named **Revoketags** (lines 19–30). Specially, when the user wants to defend against a replay attack, the value of $addr$ is associated with a count named cnt' . If the type of user is an anti-replay attack, the data owner adds **FileDelCnts** which records how many times the corresponding keyword pair (w, id) deletes and computes $addr$ by $G_{k_u}(w, cnt')$ and stores the ciphertext of $addr$ in **DelCntDiffs** (lines 25–30). Otherwise, $addr$ is computed by $G_{k_u}(w)$ (line 28). Then, D is updated by inserting a tag t computed by $SRE.Comp$ (line 29). In the last step, the data owner sends $(u, Revketags)$ and **DelCntDiffs** to the server (line 34). Then, the server stores these pairs in the **Revtag**[u] and **FileDelCnts**[u], respectively, (lines 40 and 41).

Update: Algorithm 4 describes the update algorithm, which is similar to Algorithm 3. The difference between the two Algorithms is that the file in Algorithm 4 can only be shared with the users contained in the **userList**, which can be gotten from the control list **AccessList** (lines 1–3). The data owner updates **Keyvalues** or **Revoketags** based on the op . If op is added, the data owner computes **Keyvalues** and sends **Keyvalues** to the server to add to the encrypted dataset **Dictw** (lines 7 and 8). Otherwise, the data owner updates **Revoketags** which records the deletion information (lines 11 and 12).

Search: As shown in Algorithm 5, the data user fetches the **diffs** which contains the keyword counter cnt of all keywords shared for user u from the server. Then, she gets the keyword w counter cnt . Using cnt , it generates a list of locations **TList** for these entries in **Dictw** corresponding search keyword w

Algorithm 5: Search($w, K_u, K'_u, \delta; EDB$)

```

// Data User:
1 TList = {}
2 diffs  $\leftarrow$  fetch Queues[ $u$ ] from the server
3 for each  $diff$  in diffs do
4    $(w', cnt) \leftarrow Dec_{K'_u}(diff)$ ; FileCnt[ $w'$ ] =  $cnt$ 
5 cnt  $\leftarrow FileCnt[w]$ 
6 for  $i = 1$  to cnt do
7   Tlist  $\leftarrow Tlist \cup \{G_{k_u}(w, i)\}$ 
8 revoketags  $\leftarrow$  fetch Revtag[ $u$ ] from the server
9 if  $u.type$  is anti-replay attack then
10  Delcnts  $\leftarrow$  fetch FileDelcnt[ $u$ ] from the server
11  for each  $Delcnt$  in Delcnts do
12     $(w', cnt') \leftarrow Dec_{K'_u}(Delcnt)$ ; FileDelCnt[ $w'$ ] =  $cnt'$ 
13  for each  $revtag$  in revoketags do
14     $(w', cnt') \leftarrow Dec_{K'_u}(revtag.addr)$ 
15    if  $w' == w$  &  $cnt' == FileDelCnt[w']$  then
16       $D \leftarrow revtag.D$ ;  $sk_R \leftarrow SRE.cKRev(sk, D)$ 
17    break
18 else
19  for each  $revtag$  in revoketags do
20     $w' \leftarrow Dec_{K'_u}(revtag.addr)$ 
21    if  $w' == w$  then
22       $D \leftarrow revtag.D$ ;  $sk_R \leftarrow SRE.cKRev(sk, D)$ 
23    break
24  $tkn = F(K_s, w)$ 
25 send  $sk_R$  to the server's enclave with the secure channel
26 send  $(Tlist, D, tkn)$  to the server
// Server:
27 NewInd = {}; load flag  $\leftarrow flag[u][tkn]$ 
28 for  $i = 1$  to TList.size do
29    $(ct_i, t_i) \leftarrow DictW[TList[i]]$ 
30   if  $t_i$  in  $D$  or  $i$  not in flag then
31      $ind_i = SRE.Dec(sk_R, D, ct_i, t_i)$ 
32     if  $ind_i \neq \perp$  then
33       NewInd  $\leftarrow NewInd \cup (ind_i, t_i)$ 
34     else
35       DelInd  $\leftarrow DelInd \cup \{t_i\}$ 
36     flag[ $u$ ][ $tkn$ ].add( $i$ )
37 OldInd  $\leftarrow EDB_{cache}[u][tkn]$ 
38 OldInd  $\leftarrow OldInd \setminus (ind, t) : \exists t_i \in DelInd \text{ s.t. } t = t_i$ 
39 Res  $\leftarrow NewInd \cup OldInd$ ,  $EDB_{cache}[u][tkn] \leftarrow Res$ 

```

(lines 6 and 7). If the user's type is an anti-replay attack, the data user gets the keyword revoking counter cnt' loaded from the server (lines 9–12). cnt' can be used to help defend against replay attacks (lines 15 and 16). It then generates the revoked secret key sk_R using $SRE.cKRev$ within the enclave created by SGX by loading D into the enclave (line 16). If the user's type is not an anti-replay attack, D is simply found in **revoketags** and then is used to compute sk_R using $SRE.cKRev$ within the enclave. In the last steps, the data users send $(Tlist, sk_R)$ to the server. For the server, she gets the ciphertext (ct_i, t_i) and then uses the $SRE.Dec$ to decrypt ct_i . **flag**[u] and EDB_{cache} are used to improve search performance (lines 30–36).

C. Analysis

The search communication size of X-MUSE is linear with the identifiers with the insert operation and the search result is linear with the number of identifiers that have not been deleted which is smaller than $Q-\mu SE$ [10], as the search result size of $Q-\mu SE$ is linear with the number of identifiers with inserted and deleted. On the other side, we can optimize the deletion algorithm by deleting a set of identifiers associated with w for the user u in a batch. The details of deletion optimization are

Algorithm 6: Batch-delete($K, IdList, w, del, u, \delta$)

```

// Data owner:
1 userList  $\leftarrow$  AccessList[id]
2 if  $u$  is not in userList then
3   return
4 Keyvalues  $\leftarrow \perp$ ; Cnt  $\leftarrow \perp$ 
5  $(K_u, K'_u, K_t) \leftarrow \text{UsersKeys}[u]$ 
6 if FileCnts[ $u$ ][ $w$ ] =  $\perp$  then
7    $\text{FileCnts}[u][w] = 0$ ;  $(sk, D) \leftarrow \text{SRE.KGen}(1^\lambda)$ 
8    $\text{MSK}[u][w] \leftarrow sk$ ;  $\text{D}[u][w] \leftarrow D$ 
9 D  $\leftarrow \text{D}[u][w]$ 
10 if  $u.type$  is anti-replay attack then
11   if FileDelCnts[ $u$ ][ $w$ ] =  $\perp$  then
12     FileDelCnts[ $u$ ][ $w$ ] = 0
13   for each  $id$  in IdList do
14     FileDelCnts[ $u$ ][ $w$ ] ++;  $D \leftarrow \text{SRE.Comp}(D, t)$ 
15 else
16   for each  $id$  in IdList do
17      $D \leftarrow \text{SRE.Comp}(D, t)$ 
18 if  $u.type$  is anti-replay attack then
19    $cnt' \leftarrow \text{FileDelCnts}[u][w]$ ;  $addr = \text{Enc}_{K_u}(w, cnt')$ 
20    $\text{DelCntDiffs} \leftarrow \text{DelCntDiffs} \cup \{(u, \text{Enc}_{K'_u}(w, cnt'))\}$ 
21 else
22    $addr = \text{Enc}_{K_u}(w)$ 
23  $\text{D}[u][w] \leftarrow D$ ; Revoketags  $\leftarrow \text{Revoketags} \cup (addr, D)$ 
24 send  $u, \text{Revoketags}, \text{DelCntDiffs}$  to the server // Server:
25 Insert all  $(u, \text{Revoketags})$  into Revtag[ $u$ ]
26 Insert all  $(u, \text{DelCntDiffs})$  into FileDelCnts[ $u$ ]

```

shown in Algorithm 6. As the cloud server cannot know the time of deletion, *X-MUSE* can promise backward privacy II.

V. B-MUSE: BLOCKCHAIN-BASED MUDSSE

A. Intuition and Overview

X-MUSE needs the data owner to update the compressed data structure D to the server for every keyword and every user even if the updated pairs of (w, id) are the same for the users, which causes high communication overhead between the cloud server and the data owner. In addition, *X-MUSE* is not secure against a server that provides a false result, e.g., by dropping or modifying (parts of) D . To solve this problem, we employ blockchain technology by storing D on the blockchain. Due to the huge number of deleted items, even a highly compact data structure still cannot be stored in a relatively small block and is inefficient in practice. Therefore, we need to design a suitable on-chain data structure Fig. 3 when storing D on the blockchain based on *X-MUSE*. Fig. 4 gives a high-level design of *B-MUSE*. Step 1 and step 2 are the same as *X-MUSE*. When the data owner wants to add/delete a (w, id) pair, then the data owner initializes or updates the data structure named $\mathbf{D}$ and stores $\mathbf{D}$ on the blockchain (see step 3). At step 4, when the client wants to search documents matching a query keyword, she first downloads $\mathbf{D}$ from the blockchain and generates the revoked secret key sk_r using $\mathbf{D}$ and generates the search token. At step 4, the cloud server searches the encrypted dataset and sends the search results to the user.

B. B-MUSE Solution

The *Setup* and *Enroll* algorithms are the same as *X-MUSE*, respectively.

1) *Revoke_Batch*: Algorithm 7 describes the data owner can delete a batch of (w, id) pairs for all users. First, the data

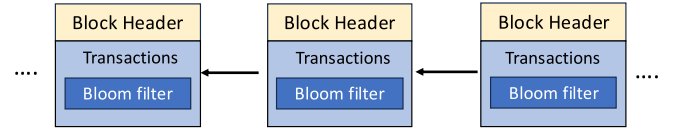


Fig. 3. Illustration of on-chain data structure.

Algorithm 7: Revoke_batch($K, w, IDList, \delta$)

```

// Data owner:
1 if batchCnts[ $w$ ] =  $\perp$  then
2   batchCnts[ $w$ ] = 0; cnt = 0
3 else
4   batchCnts[ $w$ ] ++; cnt  $\leftarrow$  batchCnts[ $w$ ]
5  $addr \leftarrow G_{k_u}(w, cnt)$ 
6 Initialize a data structure  $D$ 
7 for each  $ind$  in IDList do
8    $t \leftarrow F_{K_t}(w, ind)$ ;  $D \leftarrow \text{SRE.Comp}(D, t)$ 
9 Revoketags[ $w$ ]  $\leftarrow (addr, D)$ 
10 send Revoketags[ $w$ ] to the blockchain
// Blockchain:
11 Smart contract writes Revoketags[ $w$ ] to the blockchain

```

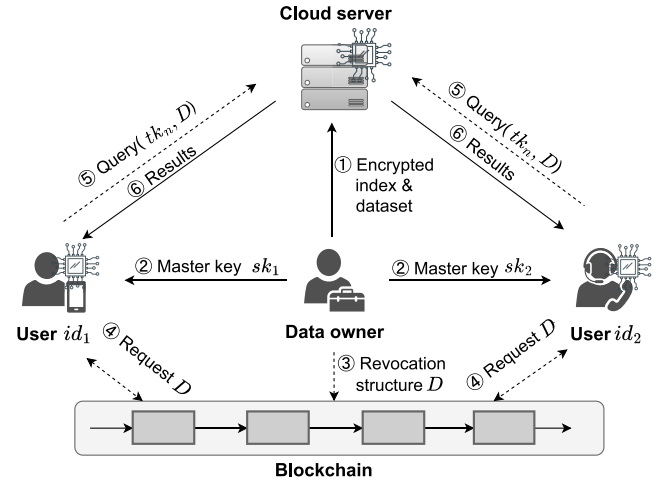


Fig. 4. Overview of B-MUSE.

owner gets the revoked keyword counter $batchCnts[w]$, which records the frequency of deletion operation on w . Then, it checks whether the counter $batchCnts[w]$ has been initialized or not. If not, the counter is set to 0. Otherwise, it increments the counter cnt (lines 1–4). Then, the data owner computes the location $addr$ on the blockchain using cnt and initializes a bloom filter D and inserts the deleted identifiers in $IDList$ associated with the keyword w . Finally, the data owner sends the $(addr, D)$ pairs to the blockchain. When receiving $(addr, D)$ pairs stored in $Revoketags[w]$, the smart contract of the blockchain stores $Revoketags[w]$ on the blockchain (line 11).

2) *Batch_Search*: Algorithm 8 describes the search process of B-MUSE. First, the data user u first gets the list $Tlist$ associated with w from the server, which is the same as Algorithm 5 (lines 1–8). Then, the data user gets the $Delcnts$ from the data owner and gets the list contained in the compressed data structure D_i from the blockchain using the smart contract (lines 9–12). The data user generates the search token tk_n using the secret key K_s and the keyword w and sends the secret key sk_w to the server's

Algorithm 8: Batch_Search($u, w, K_s, K_w, \delta; EDB$)

```

// Data user:
1  $TList = \{\}$ 
2  $diffs \leftarrow \text{fetch Queues}[u]$  from the server
3 for each  $diff$  in  $diffs$  do
4    $(w', cnt) \leftarrow Dec_{K_u'}(diff); FileCnt[w'] = cnt$ 
5    $cnt \leftarrow FileCnt[w]$ 
6   for  $i = 1$  to  $cnt$  do
7      $Tlist \leftarrow Tlist \cup \{G_{k_u}(w, i||0)\}$ 
8    $Delcnts \leftarrow \text{fetch batchCnts}[w]$  from the data owner
9    $D = \{\}$ 
10  for  $i = 1$  to  $Delcnts$  do
11     $token_i = G_{k_u}(w, i)$ 
12    smart contract :  $D_i \leftarrow \text{read}(token_i); D.append(D_i)$ 
13   $tkn = F(K_s, w)$ 
14  send  $sk_w$  to the server's enclave with the secure channel
15  send  $(Tlist, D, tkn)$  to the server
// Server:
16  $NewInd = \{\}$ 
17 for  $i = 1$  to  $TList.size$  do
18    $(ct_i, t_i) \leftarrow DictW[TList[i]]$ ;  $flag = true$ 
19   for  $d$  in  $D$  do
20     if  $t_i$  in  $d$  then
21        $DelInd \leftarrow DelInd \cup \{t_i\}$ ;  $flag = false$ 
22     break
23   if  $flag$  then
24      $ind_i = SRE.DEC((sk_w, D), ct_i, t_i)$ 
25      $NewInd \leftarrow NewInd \cup (ind_i, t_i)$ 
26  $OldInd \leftarrow EDB_{cache}[u][tkn]$ 
27  $OldInd \leftarrow OldInd \setminus (ind, t) : \exists t_i \in DelInd \text{ s.t. } t = t_i$ 
28  $Res \leftarrow NewInd \cup OldInd, EDB_{cache}[u][tkn] \leftarrow Res$ 

```

enclave by the secure channel (lines 13 and 14). Finally, the user sends the $(Tlist, D, tkn)$ to the server (line 15). For the server, the server first checks whether the ct_i under tag_i is the deleted index. If t_i is in bloom filter d , it indicates that t_i is the deleted identifier (lines 20–22).

C. Analysis

We first analyze the *correctness* of B-MUSE. The main difference between X-MUSE and B-MUSE is that the compressed data structure of D is stored on the blockchain. In general, the size of D will eventually exceed the capability of a block. We can not simply divide D into pieces and store these pieces on the blocks of the blockchain. As the tag inserted to D can cause the frequent modification of all pieces, which will cause performance degradation and the pieces of D can not be modified because the block on the blockchain cannot be rewritten. Inspired by process [19], we design a block-oriented revocation list. The data structure on the blockchain is shown in Fig. 5. Specifically, every node of the list containing D updates over a period of time can be stored in a transaction on the blockchain.

VI. SECURITY ANALYSIS

In this section, we prove the security of our schemes following the framework of game and simulation paradigm [2], [20], [21].

Theorem 1: Assuming G is a secure PRF, queues are encrypted with a semantically secure encryption scheme, and SRE is an adaptive secure scheme, X-MUSE is secure

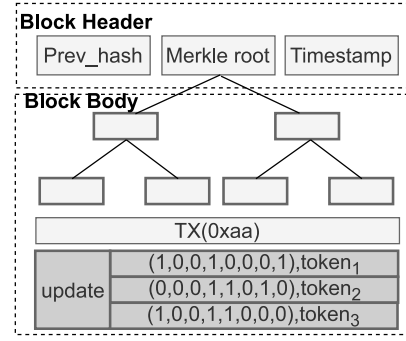


Fig. 5. Illustration of compressed data structure D on blockchain.

(Definition 1) and forward and backward private (Definitions 2 and 3) DMUSSE schemes with the following leakage:

$$\begin{aligned}
 L_H^{stp} &= \{n, m, L, ids, sizes\} \\
 L_H^{Updt} &= \{id, UpdtLeakage(u, KwList)\} \\
 L_H^{Shr} &= \{u, id, ShrLeakage(u, id, t)\} \\
 L_H^{UnShr} &= \{u, id\} \\
 L_H^{Srch} &= \{u, TimeDB(w, u), Update(w, u) \\
 &\quad SrchLeakage(w, u)\}.
 \end{aligned}$$

Proof (Sketch): The proof is conducted in the game hop like AURA [11]. To get X-MUSE simulator, we gradually replaced the real cryptographic tools used with the cryptographic tools in X-MUSE. The simulator uses L_H^{stp} to simulate the encrypted dataset and uses L_H^{Updt} or L_H^{Shr} to update the simulated dataset and uses L_H^{Srch} leakage function to retrieve the search result.

Theorem 2: Assuming G is a secure PRF, queues are encrypted with a semantically secure encryption scheme, and SRE is an adaptive secure scheme, and blockchain is secure, B-MUSE is secure (Definition 1) and forward/backward private (Definitions 2 and 3) DMUSSE schemes with the following leakage: $UpdtLeakage'(KwList)$, $ShrLeakage'(id, t)$ and $Update'(w)$ are similar with $UpdtLeakage(u, KwList)$, $ShrLeakage(u, id, t)$ and $Update(w, u)$, respectively, except that $UpdtLeakage'(KwList)$, $ShrLeakage'(id, t)$ and $Update'(w)$ apply to all users and the compact data structure D is stored on the blockchain

$$\begin{aligned}
 L_H^{stp'} &= L_H^{stp} \\
 L_H^{Updt'} &= \{id, UpdtLeakage'(KwList)\} \\
 L_H^{Shr'} &= \{id, ShrLeakage'(id, t)\} \\
 L_H^{Srch'} &= \{TimeDB(w, u), Update'(w) \\
 &\quad SrchLeakage(w, u)\}.
 \end{aligned}$$

Proof (Sketch): B-MUSE is secure for Definition 1 with leakage profiles that achieve forward and backward privacy (Definitions 2 and 3). Using L_H^{stp} , $L_H^{Updt'}$ and $L_H^{Srch'}$, the PPT simulator computes the encrypted dataset which contains the pairs (addr, value) with random values. As the (addr, value) pairs during updates and shares are computed with a PRF, thus they are indistinguishable from random values. When deleting a batch of pairs (w, id) , the data owner inserts these pairs into a bloom filter D and uploads D to the blockchain.

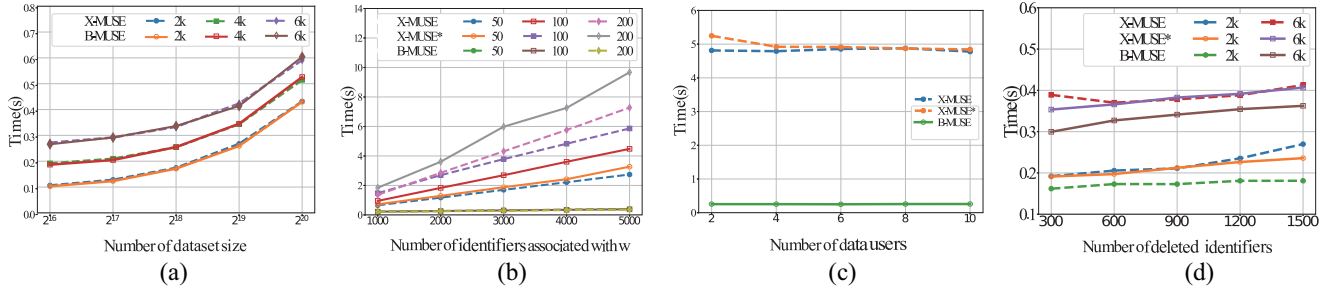


Fig. 6. Search time of *XB-MUSE*. (a) Varying dataset size. (b) Varying number of identifiers. (c) Varying number of readers. (d) Varying number of deleted identifiers.

Using $L_H^{Srch'}$, the simulator only obtains the number of previous updates for the search keyword w and when they took place. Finally, similar to $Q-\mu SE$ [10], since separate queues with different keys are used for each user and these keys are stored in intel SGX and the PRF key used for each user is also different, we manage to eliminate all cross-user leakage.

VII. EVALUATION

In this section, we first evaluate the search performance of the protocol *XB-MUSE*. Next, under the different parameter settings, we evaluate the update performance of the protocol *XB-MUSE*. Finally, we report the performance of our protocols by comparing them with previous works.

A. Experiment Setup

Setup: Our framework is implemented in roughly 4500 C++ and C LOCs. A Dell workstation with an Intel Core i7-10700 CPU run at 2.60GHZ with Ubuntu 18.04 Server, 16GB of RAM, and four threads per CPU was used to run the data user and server. Keep in mind that this computer supports the actual Intel SGX enclave, and we are using version 1.9 of the SGX driver, SDK, and platform software. The Ethereum test network TestRPC makes use of the smart contract. Web3.py serves as the foundation for the communication between Ethereum and the data owner and user.

Datasets: We use the synthetic dataset *SYN* and a real dataset Enron dataset *Enron*¹ to evaluate our protocols. Specifically, we implement a uniform workload to generate keywords as *SYN* with a uniform distribution for each document. In our experiments, we use *SYN* for our SSE schemes *X-MUSE* and *B-MUSE* and use *Enron* for our comparison experiments.

Parameter Setting: We measure our schemes' performance by evaluating the search time of *X-MUSE* and *B-MUSE*, updating the time of *X-MUSE* and *B-MUSE* and *X-MUSE** in which the user type is anti-replay attack. We set the number of the hash function is 5 if there is no special instruction. The number of leaves and the height of the GGM tree [11] which is the realization of SRE are 65536 and 16, respectively.

¹<https://www.cs.cmu.edu/~enron/>

B. Evaluation With Synthetic Datasets

1) **Search Time:** Fig. 6 shows that the search performance of the *X-MUSE* and *B-MUSE* are in the second scale. Fig. 6(a) shows the time of searching for a keyword of *X-MUSE* and *B-MUSE* that matches {2000, 4000, 6000} documents, where the database size varies from 2^{16} to 2^{20} , which search time varies from 0.10 to 0.59 s. The search performance for a keyword is acceptable. Fig. 6(b) shows the search time of a keyword ranges from 0.3 to 9.7 s where the number of associated documents varies from 1000 to 5000 when there are {50, 100, 200} deleted documents. Fig. 6(c) shows the search time of a keyword that matches 2^{16} documents by 2 to 10 readers when there are 150 deleted documents. Fig. 6(d) shows the search time of *X-MUSE*, *X-MUSE** and *B-MUSE* when the number of identifiers associated with w is 2000 and 6000 by varying the number of deleted identifiers. Fig. 6 shows that the search performance of *X-MUSE* and *X-MUSE** is similar and *B-MUSE* speeds up the search time by $2.2 \times \sim 2.4 \times$. The reason is that the computation cost of *X-MUSE** and *X-MUSE* is similar and *B-MUSE* has no operation of SRE.cKRev as the secret key sk_w is stored in the enclave of the server.

2) **Update Time:** Experimental results show that the insertion time of *X-MUSE* and *B-MUSE* are in the microsecond scale and the deletion time of *X-MUSE* and *B-MUSE* are in the millisecond scale. Fig. 7(a) indicates that the insertion time of *X-MUSE* grows with the number of keywords and is independent of the dataset size. Fig. 7(b) indicates the insertion time of *B-MUSE* with the same results as Fig. 7(a). The insertion time of *X-MUSE* and *B-MUSE* is fast where the number of associated documents varies from 2^{16} to 2^{20} when inserting 5, 10, 15, 20 keywords. Fig. 7(c) indicates the deletion time of *X-MUSE* and *X-MUSE**. Fig. 7(d) indicates the deletion time of *B-MUSE*. The deletion time of *X-MUSE*, and *X-MUSE** are practical where the number of associated documents varies from 2^{16} to 2^{20} when deleting 50, 100, 150, 200 keywords, while *B-MUSE* ranges from 1.6 ms to 15 ms as D is stored on the blockchain by the smart contract sequentially.

C. Comparison With Previous Works

1) **Search Time:** Experimental results show that the index construction time of *X-MUSE* and *B-MUSE* are in the millisecond scale. In Fig. 8(a), we evaluate the impact of the size of the deleted identifiers on the search performance of *X-MUSE* and

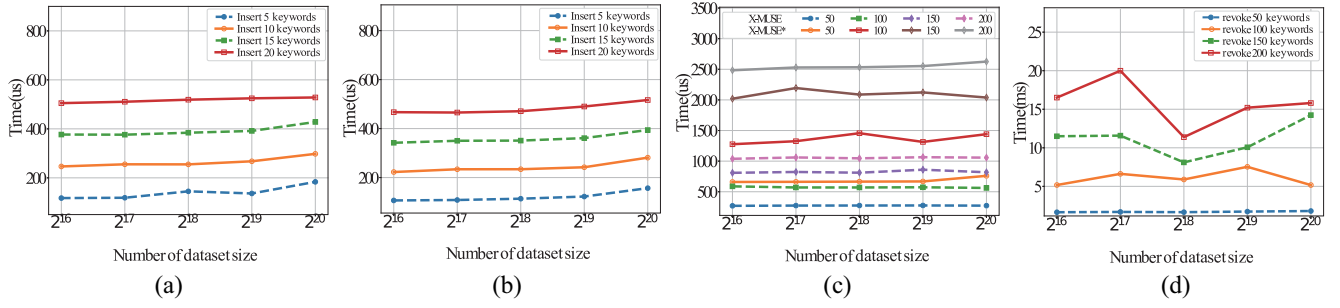


Fig. 7. Update time of XB-MUSE. (a) X-MUSE. (b) B-MUSE. (c) X-MUSE and X-MUSE*. (d) B-MUSE.

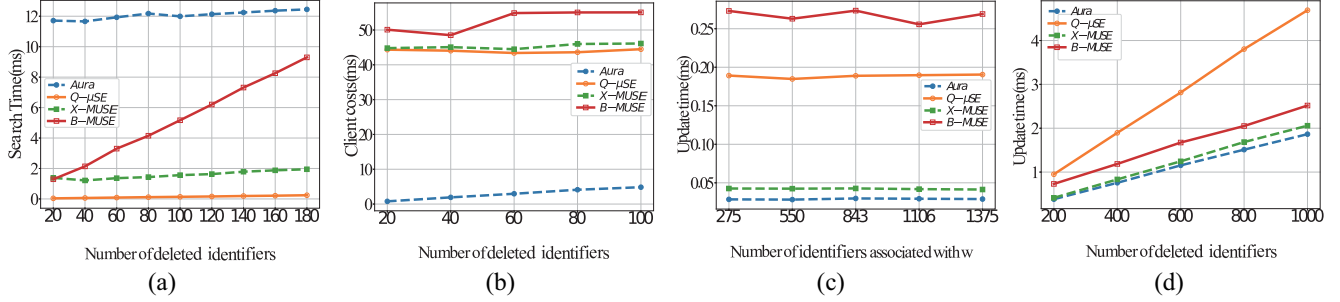


Fig. 8. Comparison with related work. (a) Varying number of searched identifiers. (b) Varying number of deleted identifiers. (c) Varying number of identifiers. (d) Varying number of deleted identifiers.

B-MUSE compared with the state of the arts on the real dataset named Enron. The number of deleted identifiers grows from 20 to 180, the search time of X-MUSE grows from 1.38 to 1.94 milliseconds and the search time of B-MUSE grows from 1.29 to 9.29 milliseconds, as shown in Fig. 8(a). The search time of X-MUSE and B-MUSE are faster than Aura. The reason is that the time for generating the revocation secret key in X-MUSE is faster than Aura. The search time of X-MUSE and B-MUSE are slower than Q-μSE. The reason is that X-MUSE and B-MUSE read items in bloom filter from the untrusted memory by OCALL provided by SGX and OCALL is an expensive operation which can be optimized by open enclave SDK.² But X-MUSE and B-MUSE have lower communication costs than Q-μSE. In Fig. 8(b), we evaluate the client costs for a keyword search by varying the number of deleted identifiers. X-MUSE and B-MUSE have more expensive costs than Q-μSE, as the clients of X-MUSE need to compute the revocation secret key and the clients of B-MUSE need to download D from the blockchain. The configuration of the bloom filter used in X-MUSE and B-MUSE is shown in Table III.

2) *Update Time*: Experimental results show that the update time of XB-Muse is in the millisecond scale. In Fig. 8(c), we observe that the update time of X-MUSE is faster than B-MUSE when the number of deleted identifiers is 40. The reason is that the client needs to load D from the blockchain. In Fig. 8(d), we show the update time for varying the number of deleted identifiers from 200 to 1000. When the keyword-associated identifiers are 4K and the number of key-value pairs is 2^{17} , the average update time of X-MUSE and B-MUSE grows from 0.41 to 2.05 milliseconds and 0.73 to 2.52

²<https://openenclave.io/sdk>

TABLE III
CONFIGURATIONS OF SEARCH COSTS

d	20	40	60	80	100
Storage(bit)	384	767	1151	1534	1918
h	13	13	13	13	13

TABLE IV
CONFIGURATIONS OF UPDATE COSTS

d	200	400	600	800	1000
Storage(bit)	3835	7669	11503	15337	19171
h	13	13	13	13	13

milliseconds, respectively, as shown in Fig. 8(d). In Table IV, we show the configurations of Fig. 8(d) and we observe that the update time of B-MUSE and X-MUSE is slower than Q-μSE, as the update procedure in Q-μSE only deletes one identifier every time. If the number of deleted identifiers is 200, the deletion related operation of Q-μSE needs to repeat 200, while B-MUSE and X-MUSE delete these identifiers in one batch.

D. Communication Size

1) *Search Communication Cost*: Experimental results show that the index communication size of X-MUSE and B-MUSE are efficient. We compare our construction communication cost with previous scheme Q-μSE [10]. With the number of documents associated with the search keyword w growing from 4000 to 20000, the search communication cost of X-MUSE grows from 106.2 to 918.73 KB and the search communication cost of Q-μSE [10] grows from 250 to 1250 KB. From Fig. 9(a), we observe that the search communication

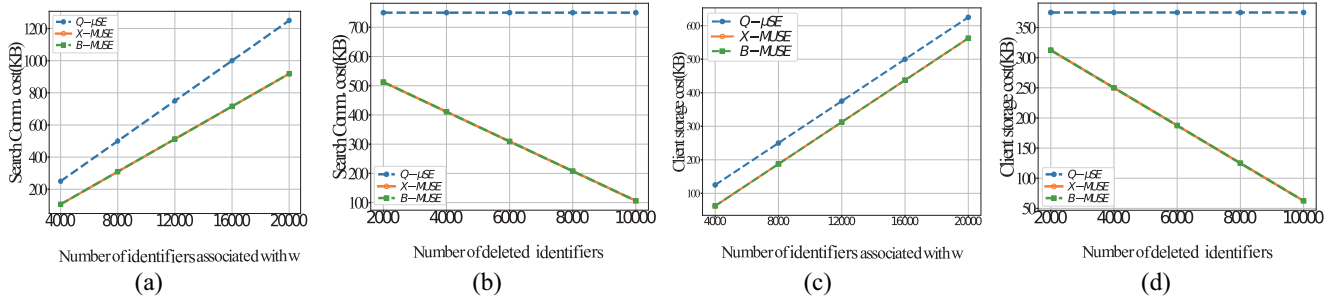


Fig. 9. Communication size. (a) Varying number of identifiers. (b) Varying number of deleted identifiers. (c) Varying number of identifiers. (d) Varying number of deleted identifiers.

cost of $B-MUSE$ and $X-MUSE$ outperforms $Q-\mu SE$ [10] by $1.3 \sim 2.4\times$. From Fig. 9(b), we observe that the search communication cost of $B-MUSE$ and $X-MUSE$ is lower than $Q-\mu SE$ by varying the number of deleted identifiers from 2000 to 10000. As the number of deleted identifiers increases, the search communication decreases as the deleted identifiers can be deleted from the server physically and our schemes do not need to send the identifiers to the client.

2) *Client Storage Cost:* Experimental results show that the client storage costs of $X-MUSE$ and $B-MUSE$ are small. When the number of identifiers associated with w ranges from 4000 to 20000 and the number of the deleted identifiers with w is fixed, the client storage cost of $X-MUSE$ and $Q-\mu SE$ grows from 62.5 to 562.5 KB and 125 to 625 KB, respectively, shown in Fig. 9(c). The reason is that our schemes do not need to store the ciphertext and the deleted identifiers associated with w . Fig. 9(d) shows that the client storage cost of $X-MUSE$ and $B-MUSE$ ranges from 312.5 to 62.5 KB by varying the number of deleted identifiers associated with a keyword w from 2000 to 10000. While the client storage cost of $Q-\mu SE$ is constant, which is 375 KB when we do not add the identifiers associated the search keyword w in the dataset. The reason is that our schemes only need to send the identifiers in plaintext associated with the search keyword w without the deleted identifiers while $Q-\mu SE$ needs to send the identifiers in ciphertext associated with the search keyword w including the deleted identifiers.

VIII. RELATED WORK

A. Multiuser Searchable Encryption

DSE has been extensively studied, both in the single-user [9], [13], [22], [23] and the multiuser setting [10], [22]. DSE schemes focus on FP and backward privacy which was initialized by Stefanov et al. [24]. FP is a secure privacy which ensures the newly uploaded files are not linked to the previous search tokens. Backward privacy is a secure privacy that ensures the newly searched token is not linked to the previously deleted files. In the single-user setting, Bost et al. first formalized the FP in [23] and backward privacy in [9] and proposed several constructions cryptographic primitives, including *Moneta* that can achieve Type-I backward privacy, *Fides* that can achieve Type-II backward privacy and *Diana_{del}* and *Janus* that can achieve Type-III backward privacy. To improve the search performance, Sun et al. [13] proposed a

practical Type -III scheme *Janus++* and a practical Type-II scheme *Aura* [11] using symmetric puncturable encryption SPE and SRE, respectively. In the multiuser setting, some solutions [25], [26] were based on Public-key encryption with keyword search (PEKS), while some solutions [8], [27] were based on symmetric encryption. Few works consider the communication overhead between the users and the storage overhead of the server and the users. XB-Muse is the first work to consider these factors for privacy in multiuser scenarios.

B. SSE Schemes With Intel SGX

The adoption of Intel SGX focused on improving the search efficiency or solving the security problems from an untrusted cloud environment when deploying the encrypted database on the cloud server [28], [29], [30], [31], [32], [33], [34]. Typically, intel SGX-based schemes use a clever combination of SGX and index structures as intel SGX can create a secure memory on the untrusted memory. Specifically, Eskandarian and Zaharia [29] and Fuhry et al. [34] combined the B+ tree with intel SGX to design the hardware-based protocols. Obliv [35] combines multimap and path ORAM with intel SGX to design the oblivious search algorithm. On the other hand, as the small size of the enclave created by intel SGX, the performance and security of SGX-based scheme are considered by the state of the arts [30], [31], [32], [33]. Specially, Orenbach et al. [31] and Taassori et al. [32] focused on designing efficient protocols by solving the performance degradation caused by the limitation of the secure memory created by the Intel SGX. Sasy et al. [30] and Lee et al. [33] focused on designing oblivious protocols against side channel attackers from the secure memory and the untrusted memory. Vo et al. [36] was proposed to design the forward and backward SE scheme using intel SGX. However, Vo et al. [36] does not protect the privacy of the multiuser scenarios. To our best knowledge, XB-Muse is the first secure scheme to efficiently support deletion operations in multiuser scenarios.

IX. CONCLUSION

To solve the problems of trust concentration and communication cost in the MUSSE, we design a practical multiuser Dynamic searchable symmetric encryption (DSSE) for adaptive revocation, called XB-Muse. First, we design a novel MUSSE scheme named $X-MUSE$ using *SRE*, and *intel SGX* as the building blocks. $X-MUSE$ can achieve the same

security level and search complexity of scheme on Q - μ SE [10]. Second, based on X -MUSE, we propose B -MUSE that achieves sublinear search efficiency and batch deletion. Third, X -MUSE and B -MUSE can achieve adaptive deletion which achieves lower storage cost and communication costs. Finally, we implement our schemes in C++ and C to evaluate our schemes. According to experimental results, our schemes outperform current approaches in terms of storage and communication overhead and have practical search efficiency.

REFERENCES

- [1] V. Pappas et al., "Blind seer: A scalable private DBMS," in *Proc. IEEE SP*, 2014, pp. 359–374.
- [2] R. Li and A. X. Liu, "Adaptively secure conjunctive query processing over encrypted data for cloud computing," in *Proc. IEEE ICDE*, 2017, pp. 697–708.
- [3] D. Cash, S. Jarecki, C. S. Jutla, H. Krawczyk, M. Rosu, and M. Steiner, "Highly-scalable searchable symmetric encryption with support for boolean queries," in *Proc. CRYPTO*, 2013, pp. 353–373.
- [4] R. Behnia, M. O. Ozmen, and A. A. Yavuz, "Lattice-based public key searchable encryption from experimental perspectives," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 6, pp. 1269–1282, Nov./Dec. 2020.
- [5] S.-F. Sun et al., "Non-interactive multi-client searchable encryption: Realization and implementation," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 452–467, Jan./Feb. 2022.
- [6] Y. Miao, R. H. Deng, X. Liu, K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 4, pp. 1667–1680, Jul./Aug. 2021.
- [7] J. Wang and S. S. M. Chow, "Omnes pro uno: Practical multi-writer encrypted database," in *Proc. USENIX Security*, 2022, pp. 2371–2388.
- [8] T. Le, R. Behnia, J. Guajardo, and T. Hoang, "MUSES: Efficient multi-user searchable encrypted database," in *Proc. USENIX Security*, 2024, p. 145.
- [9] R. Bost, B. Minaud, and O. Ohrimenko, "Forward and backward private searchable encryption from constrained cryptographic primitives," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 1465–1482.
- [10] J. G. Chamani, Y. Wang, D. Papadopoulos, M. Zhang, and R. Jalili, "Multi-user dynamic searchable symmetric encryption with corrupted participants," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 114–130, Jan./Feb. 2023.
- [11] S. Sun et al., "Practical non-interactive searchable encryption with forward and backward privacy," in *Proc. NDSS*, 2021, pp. 1–18.
- [12] V. Costan and S. Devadas, "Intel SGX explained," IACR, Bellevue, WA, USA, Rep. 86/2016, 2016.
- [13] S. Sun et al., "Practical backward-secure searchable encryption from symmetric puncturable encryption," in *Proc. CCS*, 2018, pp. 763–780.
- [14] L. Ji, J. Li, Y. Zhang, and Y. Lu, "Verifiable searchable symmetric encryption over additive homomorphism," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1320–1332, 2025. [Online]. Available: <https://doi.org/10.1109/TIFS.2025.3526062>
- [15] J. Li, L. Ji, Y. Zhang, Y. Lu, and J. Ning, "Response-hiding and volume-hiding verifiable searchable encryption with conjunctive keyword search," *IEEE Trans. Comput.*, vol. 74, no. 2, pp. 455–467, Feb. 2025. [Online]. Available: <https://doi.ieeeecomputersociety.org/10.1109/TC.2024.3485172>
- [16] C. Zhao, R. Du, K. He, J. Chen, J. Li, X. Liu, and J. Ning, "Efficient verifiable dynamic searchable symmetric encryption with forward and backward security," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 2633–2645, Feb. 2025.
- [17] P.-C. Cheng et al., "Intel TDX demystified: A top-down approach," 2023, *arXiv:2303.15540*.
- [18] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *Proc. IEEE Symp. Security Privacy*, 2015, pp. 640–656.
- [19] M. Jia et al., "PROCESS: Privacy-preserving on-chain certificate status service," in *Proc. IEEE INFOCOM*, 2021, pp. 1–10.
- [20] R. Curtmola, J. A. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," in *Proc. CCS*, 2006, pp. 79–88.
- [21] S. Kamara and T. Moataz, "Boolean searchable symmetric encryption with worst-case sub-linear complexity," in *Proc. EUROCRYPT*, 2017, pp. 94–124.
- [22] X. Song, Y. Zheng, J. Bai, C. Dong, Z. Liu, and E. Chang, "DISCO: Dynamic searchable encryption with constant state," in *Proc. ASIA CCS*, 2024, pp. 1724–1738.
- [23] R. Bost, "Σ₀φ₀Σ: Forward secure searchable encryption," in *Proc. CCS*, 2016, pp. 1143–1154.
- [24] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable encryption with small leakage," in *Proc. NDSS*, 2014, pp. 1–15.
- [25] J. Wang and S. S. M. Chow, "Unus pro omnibus: Multi-client searchable encryption via access control," in *Proc. NDSS*, 2024, pp. 1–18.
- [26] M. Wang, Z. Chen, Y. Miao, H. Huang, C. Wang, and X. Jia, "Cross-user leakage mitigation for authorized multi-user encrypted data sharing," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1213–1226, 2024.
- [27] X. Wang, J. Wu, and J. Wang, "One for all: Efficient verifiable dynamic multi-user searchable encryption in the presence of corrupted users," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 35, no. 8, 2023, Art. no. 101703.
- [28] P. Wu, Q. Shen, R. H. Deng, X. Liu, Y. Zhang, and Z. Wu, "ObliDC: An SGX-based oblivious distributed computing framework with formal proof," in *Proc. AsiaCCS*, 2019, pp. 86–99.
- [29] S. Eskandarian and M. Zaharia, "ObliDB: Oblivious query processing for secure databases," *Proc. VLDB Endow.*, vol. 13, no. 2, pp. 169–183, 2019.
- [30] S. Sasy, S. Gorbunov, and C. W. Fletcher, "ZeroTrace: Oblivious memory primitives from Intel SGX," in *Proc. NDSS*, 2018, pp. 1–15.
- [31] M. Orenbach, P. Lifshits, M. Minkin, and M. Silberstein, "Eleos: Exitless OS services for SGX enclaves," in *Proc. EuroSys*, 2017, pp. 238–253. [Online]. Available: <https://doi.org/10.1145/3064176.3064219>
- [32] M. Taassori, A. Shafiee, and R. Balasubramanian, "VAULT: Reducing paging overheads in SGX with efficient integrity verification structures," *ACM SIGPLAN Notices*, vol. 53, no. 2, pp. 665–678, 2018.
- [33] S. Lee, M. Shih, P. Gera, T. Kim, H. Kim, and M. Peinado, "Inferring fine-grained control flow inside SGX enclaves with branch shadowing," in *Proc. USENIX Security*, 2017, pp. 557–574.
- [34] B. Fuhry, R. Bahmani, F. Brasser, F. Hahn, F. Kerschbaum, and A. Sadeghi, "HardIDX: Practical and secure index with SGX," in *Proc. DBSec*, 2017, pp. 386–408.
- [35] P. Mishra, R. Poddar, J. Chen, A. Chiesa, and R. A. Popa, "Oblix: An efficient oblivious search index," in *Proc. IEEE SP*, 2018, pp. 279–296.
- [36] V. Vo, S. Lai, X. Yuan, S. Sun, S. Nepal, and J. K. Liu, "Accelerating forward and backward private searchable encryption using trusted execution," in *Proc. ACNS*, 2020, pp. 83–103.



Qin Jiang received the M.S. degree in computer science and technology from Zhengzhou University, Zhengzhou, China, in 2016, and the Ph.D. degree from Xi'an Jiaotong University, Xi'an, China, in 2022.

She was a Ph.D. intern student with the National University of Singapore, Singapore, from April 2020 to March 2021. She is currently working with the Engineering Research Center of Digital Forensics, Ministry of Education, Nanjing University of Information Science and Technology, Nanjing, China. Her research interests include searchable encryption, applied cryptography, and blockchain.



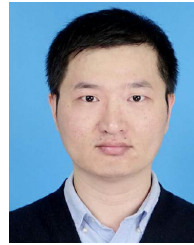
Xu Yang received the B.S. degree in information security from the University of Electronic Science and Technology of China, Chengdu, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Xi'an Jiaotong University, Xi'an, China.

His research interests include blockchain technology and searchable encryption.



Sifan Chen received the B.S. degree in computer science and technology from Xi'an Jiaotong University, Xi'an, China, in 2022, where he is currently pursuing the M.S. degree with the School of Computer Technology.

His research interests include blockchain technology and searchable encryption.



Fuyuan Song received the Ph.D. degree in software engineering from Hunan University, Changsha, China, in 2021.

From September 2019 to September 2020, he was a visiting Ph.D. student with the BBCR Lab, Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada. He is currently an Assistant Professor with the School of Computer Science, Nanjing University of Information Science and Technology, Nanjing, China. His research interests include cloud computing, applied cryptography, privacy protection, and blockchain.



Zhangjie Fu (Member, IEEE) received the Ph.D. degree from the School of Information Science and Engineering, Hunan University, Changsha, China, in 2012.

He is a Visiting Scholar with the Department of Computer Science and Engineering, State University of New York at Buffalo, Amherst, NY, USA, from 2015 to 2016. He is currently a Professor and the Dean with the School of Computer and Cyberspace Security, Nanjing University of Information Science and Technology, Nanjing, China, where he is also

filling the post of the Director of Engineer Research Center of Digital Forensic, Ministry of Education. His research interests include digital forensics, blockchain security, and artificial intelligence security.



Saiyu Qi received the B.S. degree in computer science and technology from Xi'an Jiaotong University, Xi'an, China, in 2008, and the Ph.D. degree in computer science and engineering from Hong Kong University of Science and Technology, Hong Kong, in 2014.

He is currently an Associate Professor with the School of Computer Science and Technology, Xi'an Jiaotong University. His research interests include applied cryptography, cloud security, distributed system, and pervasive computing.